

Workload Specification for Utilizing VMP 3.5.1 with CIA-toolchain and ProMBA

Written by Jeffrey

April 13, 2025

Abstract

This document specifies a comprehensive workload for utilizing VMP 3.5.1 (Virtual Machine Protector) in both standalone mode and in combination with the CIA-toolchain integrated with the ProMBA MBA deobfuscation system. The workload covers the entire chain including obfuscation, test binary generation, dynamic trace extraction, IR lifting, symbolic execution, and automatic MBA expression simplification. This specification is intended for researchers or practitioners who wish to assess the integration or operate the tools together to analyze and reverse engineer obfuscated binaries.

1 Introduction

VMP 3.5.1 is a commercial VM-based obfuscator widely used to protect software by transforming simple instructions into complex, virtualized code. Meanwhile, the CIA-toolchain is an automated framework that leverages chosen-instruction attacks to extract the semantics of virtualized code, while ProMBA is designed to simplify intricate Mixed Boolean-Arithmetic (MBA) expressions. In our proposed workload, we describe:

- How to set up and utilize VMP 3.5.1 to obfuscate test binaries.
- A pipeline to generate obfuscated binaries and extract execution traces.
- Integration of CIA-toolchain with ProMBA to simplify the MBA expressions extracted from VMP-obfuscated handlers.
- Procedures for testing these tools *together* in an integrated workflow or *separately* as standalone components.

2 Workload Overview

The complete workload is divided into four phases:

Phase 1: Obfuscation Setup with VMP 3.5.1. Prepare and configure VMP 3.5.1 to obfuscate a set of plain binaries.

Phase 2: Test Binary Generation and Execution. Generate and execute obfuscated binaries while capturing execution traces using dynamic instrumentation.

Phase 3: Analysis with CIA-toolchain + ProMBA Integration. Process the captured traces: perform IR lifting, symbolic execution, and utilize ProMBA to simplify extracted MBA expressions.

Phase 4: Standalone Analysis and Comparative Evaluation. Run CIA-toolchain without ProMBA, and ProMBA in isolation on pre-extracted MBA expressions, to compare performance and simplification quality.

3 Phase 1: Obfuscation Setup with VMP 3.5.1

3.1 Installation and Configuration

- Install VMP 3.5.1 on a dedicated analysis machine.
- Set up configuration files for VMP to use default settings as well as additional options (e.g., enabling mixed boolean-arithmetic obfuscation).
- Prepare a set of sample binaries (preferably simple C programs) that include varied arithmetic and logical operations.

3.2 Obfuscation Process

1. Use VMP 3.5.1 to obfuscate each sample binary. For example, run:

```
vmp.exe -input sample.exe -output sample_vmp.exe -mode full
```

2. Verify that the output binaries include virtualized code blocks and MBA-obfuscated expressions (check for characteristic VM dispatch loops and complex arithmetic patterns).

4 Phase 2: Test Binary Generation and Execution

4.1 Dynamic Trace Extraction

- Prepare an instrumentation environment (using Intel PIN or a similar tool).
- Execute the VMP-obfuscated binary and capture the execution trace; focus on regions demarcated by known *anchor* instructions (which can be injected prior to obfuscation).
- Save the trace logs for subsequent analysis.

4.2 Sample Workload Metrics

Record the following metrics for each binary:

- Binary size (before and after obfuscation).
- Number of virtualized handlers extracted.
- Trace length (number of instructions logged).

5 Phase 3: Analysis with CIA-toolchain + ProMBA Integration

5.1 CIA-toolchain Processing

- Use the CIA-toolchain to process the captured traces:
 1. **Trace Slicing:** Isolate the portions corresponding to virtualized instruction handlers.
 2. **IR Lifting:** Convert these sliced traces into an intermediate representation.
 3. **Symbolic Execution:** Derive symbolic formulas that capture the semantics of the obfuscated handlers.

5.2 Invoking ProMBA

- Convert the symbolic formula(s) into a C expression file:

```
// Example: target_expr.c
#include <stdint.h>
uint32_t target_expr(uint32_t x, uint32_t y) {
    uint32_t out;
    // Insert the extracted complex MBA expression here
    out = ((x ^ 0xBEEF) + ((x & 0xBEEF) << 1)) ^ (x - y); %%
    return out;
}
```

- Call ProMBA:

```
./promba -timeout 60 target_expr.c
```

- Parse the output from ProMBA. Expected output is a simplified expression (for instance, `out = x - y`) along with the rewriting rule summary and metrics (such as reduction in AST nodes).

5.3 Verification and Mapping Rule Recording

- Optionally, verify using an SMT solver (e.g., Z3) that the ProMBA-simplified expression is equivalent to the original symbolic expression.
- Record the final mapping rule for the virtual handler, such as: `VM.handler: (x - y)`.

6 Phase 4: Standalone Analysis and Comparative Evaluation

6.1 Running CIA-toolchain Alone

- Process the same set of VMP-obfuscated binaries using the standard CIA workflow (without invoking ProMBA) and document:
 - The size of symbolic expressions produced.
 - The time taken for symbolic execution and SMT equivalence checking.
 - Any cases where the solver timed out or failed to simplify.

6.2 Running ProMBA Separately

- Isolate MBA-obfuscated expressions (from VMP binaries) from the trace using CIA’s trace slicer.
- Run ProMBA directly on these expressions and measure:
 - Simplification success rate.
 - AST node count reduction.
 - Time taken per expression.

6.3 Performance Comparison

Prepare a comparative table such as:

Table 1: Comparison Metrics for CIA-toolchain with and without ProMBA			
Metric	CIA-only	ProMBA-Integrated	ProMBA-only
Average Expression Size (nodes)	250	15	10
Average SMT Check Time (s)	120	30	—
Overall Mapping Rule Success Rate	75%	90%	—
Total Processing Time per Binary (s)	300	350	—

The table should reflect that while integrating ProMBA adds some overhead, it significantly reduces the expression size and improves mapping rule accuracy and verification success.

7 Conclusion and Final Workload Summary

In this workload, we have detailed the following:

1. **Setting up VMP 3.5.1:** Installing and configuring VMP 3.5.1 to obfuscate test binaries.
2. **Dynamic Trace Extraction:** Executing the obfuscated binaries and capturing execution traces with an instrumentation tool.

3. **CIA-toolchain Processing:** Using CIA to slice traces, lift them to IR, and perform symbolic execution to obtain complex MBA formulas.
4. **ProMBA Integration:** Exporting extracted formulas to a C file, invoking ProMBA, and incorporating the simplified output into the mapping rule database.
5. **Comparative Evaluation:** Running the tools separately as well as together to compare performance metrics such as AST node reduction, time efficiency, and success rate.

This integrated workload enables automated analysis of VMP 3.5.1 obfuscated binaries. It improves the clarity and verification of the virtualized instruction semantics, making further tasks (like devirtualization and exploit generation) both feasible and efficient.

Notes:

- Ensure that all tool dependencies (VMP 3.5.1, CIA-toolchain, ProMBA, SMT solvers) are correctly installed.
- Adapt configuration parameters (e.g., timeout values, synthesis depths) based on the complexity of the test binaries.
- Use containerization (e.g., Docker) if necessary to encapsulate the environment and reduce setup complications.

References: The workload references documentation from <https://github.com/chosen-instruction>, CIA-toolchain and its accompanying documentation at <https://github.com/chosen-instruction-a>, CIA-doc-en, as well as published details on ProMBA (Lee et al., 2023).